COMP3711 Midterm Cheat Sheet — Fri Oct 16, 20:00–22:00, LTC (L2) — coverage: prerequisites → binary search → mergesort/inversions → max subarray → multiplication → randomization → quicksort/selection → heaps → lower bounds & linear sorting. Answer = idea · pseudocode (define I/O) · (correctness) · running time.

1. Asymptotics

f=O(g): ∃c,n$_0$: f(n)≤c·g(n) ∀n≥n$_0$. **Ω**: f≥c·g. **Θ**: both (prove each separately). **o**: f/g→0. Worst-case Θ(g) = O(g) on all inputs + one input family with Ω(g).

Ladder: 1 < loglog n < log n < (log n)^c < n^ε (any ε>0) < n < n log n < n^1.5 < n² < n^c < 2^(√n) < 2^n < 3^n < n! < n^n. Polylog < any polynomial; polynomial < any exponential c^n (c>1).

Rules: log base irrelevant (log_b a = log a/log b). log(n^c)=Θ(log n). a^(log_b n) = n^(log_b a): 3^(log$_2$n)=n^1.585, 4^(log$_2$n)=n², 16^(log$_4$n)=n², 2^(loglog n)=log n. Exponent: **added** const same class (2^(n+5)); **multiplied** const different (2^(2n)=4^n ≠ O(2^n); 5^n=o(7^n); 2^n vs 2^(n/2): B faster). n! = Θ(√n (n/e)^n); log(n!) = Θ(n log n); n! ≥ (n/2)^(n/2); n^(loglog n) = (log n)^(log n) (take logs). Weird pairs: substitute m=log n or m=√n.

Sums: Σ_{i≤n} i^c = Θ(n^(c+1)) (upper: n·max term; lower **half trick**: top half, each ≥ mid). Σ i·log i = Θ(n² log n). Σ √i·n = Θ(n^2.5). **H_n = Σ1/i = Θ(log n)** (doubling blocks, each block in [½,1], log n blocks). Σ_{i≥0} r^i: r<1 → Θ(1) (=1/(1−r)); r>1 → Θ(r^k) last term; r=1 → k+1. Σ i/2^i = 2. Σ_{i=k+1}^{N} 1/i = ln(N/k)+O(1). (1−1/n)^n ≈ 1/e ≈ 0.37.

A / B / U table

Reduce to dominant term, logs base 2. **U** iff a one-sided bound (only O or only Ω) leaves the order undecided (e.g. O(n²) vs Θ(n log n): U; Ω(n³) vs O(n^2.81): B; O(polylog) vs Ω(n^0.1): A — bound still forces it). Same Θ class → U ("always faster" fails). Σ-expressions and recurrences: solve first (4T(n/4)+n = n log n; 5T(n/2)+n² = n^2.32; 2T(n/2)+n = n log n; 3T(n/2)+n = n^1.585).

2. Recurrences

Master (T=aT(n/b)+n^d): a>b^d → Θ(n^(log_b a)) leaves; a=b^d → Θ(n^d log n); a<b^d → Θ(n^d) root. Extra log in f: same case ⇒ one more log power (4T(n/4)+n log n = n log² n). **Leaves = a^(log_b n) = n^(log_b a).**

Expansion (write this): T(n)=aT(n/b)+f(n)=a²T(n/b²)+a f(n/b)+f(n)=…=a^k T(n/b^k)+Σ_{i<k} a^i f(n/b^i). Stop k=log_b n. Level i cost a^i f(n/b^i): same each level → ×log_b n; geometric ↓ → Θ(f(n)); geometric ↑ → Θ(last)=Θ(leaves). **Tree table:** level i: a^i nodes, size n/b^i, cost/node f(n/b^i), level total; leaves n^(log_b a)·T(1). **√n:** n=2^m, S(m)=T(2^m): T(√n)→S(m/2). **Induction:** guess, prove T(n)≤cn^α with the same c.

T(n)=	Θ	T(n)=	Θ
T(n/2)+1	log n	2T(n/2)+n	n log n
2T(n/2)+√n	n (leaves)	2T(n/2)+n log n	n log² n
3T(n/5)+2n	n	3T(n/4)+n	n
3T(n/3)+n	n log n	3T(n/2)+n	n^1.585
4T(n/2)+n	n²	4T(n/2)+n²	n² log n
7T(n/2)+n²	n^2.81	8T(n/2)+n²	n³
64T(n/4)+n³	n³ log n	27T(n/3)+n²	n³
9T(n/3)+n²	n² log n	10T(n/3)+n²	n^(log$_3$10)
4T(n/4)+n log$_4$n	n log² n	2T(n/4)+√n	√n log n
2T(n/4)+n	n	T(n/8)+1	log n
2T(√n)+log n	log n·loglog n	T(√n)+log n	log n
T(n/2)+log n	log² n	T(n/2)+n	n
T(n−1)+n	n²	2T(n−1)+1	2^n
T(αn)+T(βn)+n, α+β=1	n log n	same, α+β<1	n
T(n/4)+T(3n/4)+n	n log n	T(n/4)+T(3n/4)+1	n

3. Binary search & D&C designs

```
BS(A,l,r,x): if l>r nil; m=⌊(l+r)/2⌋;
A[m]=x→m;
  x<A[m]→BS(l,m-1) else BS(m+1,r)
T=T(n/2)+1=log n
LowerBound(x): l=1,r=n+1; while l<r:
m=⌊(l+r)/2⌋;
  if A[m]<x l=m+1 else r=m; return l  (first
≥x; "≤" → first >x)
count(x)= first>x − first≥x.   Rotated:
A[m]>A[m+1]→m;
  A[m]≥A[l]→l=m+1 else r=m-1.  Bitonic peak:
A[m]<A[m+1]→right else left.
LastOcc(A,p,r,x) [A[p]=x]: if p=r or A[r]=x
ret r; q=mid;
  A[q]=x&A[q+1]>x→q; A[q]=x→Last(q+1,r); else
Last(p,q-1)
Distinct values (Θ(log n) many): repeat
LastOcc from each
  block start → O(log² n).   Galloping:
1,2,4,…→O(log k).
```

k-th smallest of 2 sorted arrays O(log k): compare A[k/2], B[k/2]; smaller one + its prefix have rank<k → discard k/2 from that side; recurse with k/2 (missing entry = ∞); k=1: min of heads.

Template: Name(input)→output; base case; split; recurse; combine (say exactly); T(n)=aT(n/b)+split+combine; solve; one-line why combine is right. **Hints in target bound:** log²n ⇒ log n binary searches; log k ⇒ halve k; n log log n ⇒ blocks of size log n; k log k ⇒ heap of size k; n expected ⇒ random pivot/good-pivot; n+k log k ⇒ linear pass then sort k items; n log k ⇒ k-way heap or recursion depth log k.

Mergesort 2T(n/2)+n; merge: two pointers, ∞ sentinels, ties take left (stable), Θ(n) always. **Inversions**: during merge, when right element placed add #remaining left. **Max subarray**: n³ → n² (reuse sums) → n log n (best suffix of L + best prefix of R) → **Kadane** cur=max(A[i],cur+A[i]). Max product: track max&min. Stock: diffs then Kadane. **Karatsuba** 3 mults: $(a_1+a_0)(b_1+b_0)-a_1b_1-a_0b_0$, n^1.585. **Strassen** 7 products n^2.81 (leaves dominate ⇒ only a matters). **Closest pair**: split by x, strip 2d sorted by y, O(1) nbrs, n log² n (n log n presorted). **Majority / >n/3**: candidates of halves (≤1 / ≤2 each), count ≤4 in O(n), 2T(n/2)+n = n log n; Boyer-Moore O(n): c=0 adopt, equal c++, else c−−, verify. **Grid local min O(n)**: min of border+middle row+col; if not local min recurse into quadrant with smaller nbr; T(n/2)+n. **k-sort into k blocks O(n log k)**: median (linear select) partition, recurse, stop at size n/k: 2T(n/2)+n with T(n/k)=1. **k-near-sorted**: stack pass removes ≤2k, sort k log k, merge. **Towers of Hanoi** 2^n−1.

4. Sorting & selection

```
Partition(A,l,r): x=A[r]; i=l-1; for j=l..r-1:
if A[j]≤x: i++,swap(i,j)
  swap(i+1,r); return i+1.  Inv: A[l..i]≤x |
A[i+1..j-1]>x | ? | x
QS: m=Partition; QS(l,m-1); QS(m+1,r). best
2T(n/2)+n; worst T(n-1)+n=n²
  (sorted input, last pivot); random pivot:
E=Θ(n log n) on EVERY input.
3-way (ties): lt,i,gt; A[i]<p:
swap(lt,i),lt++,i++; >p: swap(i,gt),gt--;
  =p: i++. Loop while i≤gt. All-equal array:
Lomuto → n².
Select(A,l,r,k): rand pivot→m; q=m-l+1;
k=q→A[m]; k<q→left,k; else right,k-q
Fisher-Yates: for i=1..n swap A[i],
A[Random(1,i)]  (uniform, O(n))
CountingSort keys 1..k: C[v]++; prefix sums;
for j=n..1: B[C[A[j]]]=A[j];
  C[A[j]]--  (backwards ⇒ stable) Θ(n+k).
Radix: LSD digit by digit,
  stable counting sort each, Θ(d(n+k)); base n
⇒ ints ≤ n^c sort in Θ(cn).
```

QS proof: z_1<…<z_n sorted; z_i,z_j compared ⇔ first pivot from {z_i..z_j} is z_i or z_j ⇒ Pr=2/(j−i+1); E=Σ_{i<j}2/(j−i+1) ≤ Σ_i 2H_n = O(n log n). With ties: ≤2/|Z'_ij|. **Select:** good pivot (middle half) prob ½, removes ≥¼; E[tries per good]=2; size ≤(¾)^i n after i good; E ≤ Σ 2(¾)^i n = 8n = O(n). Indicator version: cases i≤k≤j: 2/(j−i+1); i<j<k: 2/(k−i+1); k<i<j: 2/(j−k+1); each sums O(n). Deterministic median (black box) ⇒ T(n/2)+n = O(n); weighted median: partition at median, recurse on heavy side.

Sort	Worst	Notes	Space	Stable
Insertion	n²	O(n+I), I inversions; sorted: n; min comparisons on sorted	1	Y
Selection	n²	always n², n swaps	1	N
Merge	n log n	always; recursive; optimal	n	Y
Heap	n log n	always; in place; optimal	1	N
Quick(rand)	n²	E n log n; recursive; fastest in practice	log n	N
Counting	n+k	ints in [1,k]	n+k	Y
Radix	d(n+k)	d digits base k	n+k	Y

Radix linear iff w=log U = O(log n) iff range U = poly(n); T=Θ(⌈w/log k⌉(n+k)), best base k=n. [0,n³): 3 passes Θ(n). [1,n^(ln n)]: w=Θ(log²n) ⇒ Θ(n log n), no better than comparison; k=U ⇒ super-poly. Fractions p/q, p,q≤n in O(n): gap ≥1/n² ⇒ key K=⌊n²p/q⌋∈[n,n³], radix base n (no gcd, no floats). Bucket sort: uniform [0,1), Θ(n) average. Pick: ints in [1,n]→counting Θ(n); ints ≤n²→radix base n Θ(n); few inversions / each ≤d away→insertion O(n+I) or heap of size d+1 O(n log d); stable n log n→merge; in-place n log n→heap; general→quicksort. Range queries (x,y] count: sort, two LowerBound → O(log n) after O(n log n); with ints ≤k: counting array C, answer C[y]−C[x] in O(1). Strings total length n: bucket by length, LSD from longest, O(n) (padding → Ω(n²)).

5. Randomization

Recipe: X=ΣX_i (indicators or waiting times); E[X_i]=Pr[X_i=1] or 1/p; add (**linearity, no independence needed**); simplify. E[XY]=E[X]E[Y] only if independent. Tail: E[X]=Σ_{i≥1}Pr[X≥i]. Geometric p: E=1/p. Measure = expected time on worst-case input. Both bounds: max-term up, half-trick down.

Classic	Decomposition → answer
Hiring	Pr[i-th hired]=1/i → H_n=Θ(log n); exactly 1 hire 1/n; n hires 1/n!
Hat check / fixed pts	each 1/n → E=1
Coupon all n	stage i waits n/(n−i) → nH_n=Θ(n log n)
Coupon first n/2	n(1/n+…+1/(n/2+1)) ∈ [n/2, n] → Θ(n)
Coupon last half	n·H_{n/2} = Θ(n log n); last log n: Θ(n loglog n)
Distinct after n boxes	miss type i: (1−1/n)^n → n−n(1−1/n)^n ≈ 0.63n
Empty slots, n into n	n(1−1/n)^n ≈ n/e; exactly-one: n(1−1/n)^(n−1); lonely key: (1−1/n)^(n−1)
Birthday / lockers	pairs collide 1/m each → C(n,2)/m; ≈1 at n≈√(2m)
Hashing chaining	α=n/m, expected O(1+α); m=Θ(n) ⇒ O(1)
Reservoir k of N	item t in sample: k/t at step t, k/N final; ever stored k(1+H_N−H_k)=Θ(k log(N/k)); not independent
Random probe search (F25)	binary search with random record; good call ½, removes ¼; cost ≤2nΣ(¾)^i=8n
Uniform {0,1,2} from fair coin	2 bits, reject 11; E[flips]=2·(1/(3/4))=8/3
Try-and-check majority	success >½ → <2 rounds, O(n) exp; top-10% pick: 10 scans

6. Heaps

Array H[1..n]: children 2i, 2i+1; parent ⌊i/2⌋; complete, height ⌊log n⌋. **Insert**: append, bubble up (swap with parent while smaller) O(log n). **Extract-min**: last→root, bubble down (swap with smaller child while larger) O(log n). Find-min O(1). Decrease-key: bubble up; increase-key: bubble down. Delete arbitrary: swap with last, fix both ways (need position map). **Build-heap O(n)**: bubble down i=⌊n/2⌋..1; height-h nodes ≤ n/2^(h+1), cost O(h): Σ h·n/2^(h+1) ≤ n·Σh/2^h/2 = n. F25 version: merge pairs of (2^k−1)-heaps under new root, O(k) each; Σ_k k·2^(m−k−1) ≤ n+1. **Heapsort** n log n in place, not stable. Drawing: after every insert write array + tree.

d-ary heap: children of i: d(i−1)+2..di+1; parent of j: ⌊(j−2)/d⌋+1; height log_d n; insert O(log_d n); extract O(d log_d n); big d when decrease-keys ≫ extracts; d=n ⇒ sorted array. **Sliding-window max**: max-heap of (val,idx), lazy-delete expired roots: each item in/out once, but heap may reach Θ(n) (increasing input) ⇒ O(n log n)/O(n); with position map eager delete ⇒ O(n log W)/O(W); monotonic deque O(n) worst (new elem pops smaller from back; front = max).

Designs: k-way merge N items: heap of k heads, O(N log k), O(k) space. Running median: max-heap low / min-heap high, sizes differ ≤1, insert O(log n) median O(1). k-th smallest in read-only heap: aux heap seeded root, pop & push children, O(k log k). Report all <x: DFS from root, prune at ≥x, O(k) (≤2k+1 calls). Largest ⌈log n⌉ in n loglog n: blocks of log n sorted (n/log n · log n loglog n), heap-merge tops log n times. Min over m sets with add O(log m+log n_i): per-set heap + top heap of minima + back pointers updated on swaps. d-disordered sort: heap of size d+1, O(n log d). Huffman/optimal merge order: repeatedly merge two smallest (heap), Θ(n log n).

7. Decision trees & lower bounds

Comparison algorithm = binary tree; worst case = height ≥ **log$_2$(#outputs)**. Sorting: n! outputs ⇒ log n! = Ω(n log n) (n! ≥ (n/2)^(n/2)). k smallest in order: C(n,k)k! = n(n−1)…(n−k+1) ≥ (n/2)^k ⇒ Ω(k log n). Merge two sorted n-arrays: C(2n,n) ≥ 2^n ⇒ Ω(n). Max/min need n−1; both ≈3n/2. Median of 3 in ≤3 comparisons; max of 3 in 2. Draw: branch on each comparison the algorithm makes, leaves = outputs (recursive min of 4: a_1:a_2, a_3:a_4, winners).

8. Pitfalls (lose points)

Give Θ with α,β when asked. Include leaf level. No Master when forbidden. Define subroutine I/O. Justify every running time. Don't assume independence. Draw every intermediate heap. Both bounds when asked (upper via max term, lower via half trick). U only when a one-sided bound truly leaves it open. Exponent n vs n−1 in (1−1/n) (self excluded ⇒ n−1).